

Pixtall – User Credit System Design

Plan	Price	Credits	Standard(Image gen cost)	Premium(Image gen cost)
Pay as you go	₹499 one-time	500	10 cr	20 cr
Creator	₹999/month	1,200	8 cr	16 cr
Pro	₹1,999/month	2,600	7 cr	14 cr
Business	₹3,999/month	6,000	6 cr	12 cr

Generation Job Tracking

Every image-generation request should receive a unique **Job ID**.

The job should store the user, image type, number of images, required credits, reserved credits and generation status.

The lifecycle should be:

Created → **Credits Reserved** → **Processing** → **Completed / Failed**

The Job ID should also be linked with the credit transaction so the exact credit usage of every generation can be traced.

GST should be handled separately by the billing system.

Final Price = Plan Price + 18% GST

Plan	Base Price	18% GST	Final Payment
Pay as you go	₹499	₹89.82	₹588.82
Creator	₹999	₹179.82	₹1,178.82

Pro	₹1,999	₹359.82	₹2,358.82
Business	₹3,999	₹719.82	₹4,718.82

Overall Flow

Purchase Plan → Payment + GST → Verify Payment → Add Credits → Generate → Check/Reserve Credits → Process Image → Consume/Return Credits → Update Wallet → Record Transaction

Schema:

1. Users

Column	Type	Key	Description
id	UUID	PK	Unique user ID
email	VARCHAR	UNIQUE	User email
name	VARCHAR	User name	
created_at	TIMESTAMP	Account creation time	

2. Subscriptions

Column	Type	Key	Description
id	UUID	PK	Subscription record ID
user_id	UUID	FK → users.id	User
plan	VARCHAR	Pay as you go / Creator / Pro / Business	
payment_id	VARCHAR	Payment provider transaction ID	
payment_status	VARCHAR	Pending / Paid / Failed	

base_amount	DECIMAL	Price before GST	
gst_amount	DECIMAL	18% GST amount	
total_amount	DECIMAL	Final amount paid	
status	VARCHAR	Active / Expired / Cancelled	
start_date	TIMESTAMP	Plan activation date	
renewal_date	TIMESTAMP	Next renewal date; null for one-time plan	
created_at	TIMESTAMP	Record creation time	

3. Wallet

Column	Type	Key	Description
id	UUID	PK	Wallet ID
user_id	UUID	FK → users.id, UNIQUE	Wallet owner
available_credits	INTEGER	Credits available to spend	
reserved_credits	INTEGER	Credits currently locked for active jobs	
updated_at	TIMESTAMP	Last wallet update	

4. Generation jobs

Column	Type	Key	Description
id	UUID	PK	Unique Job ID
user_id	UUID	FK → users.id	User who requested generation
image_type	VARCHAR	Standard / Premium	
image_count	INTEGER	Number of images requested	
credit_cost	INTEGER	Total calculated credit requirement	
reserved_credits	INTEGER	Credits reserved for this job	
consumed_credits	INTEGER	Credits finally consumed	

status	VARCHAR	Created / Queued / Processing / Retrying / Completed / Failed	
provider_request_id	VARCHAR	Request/reference ID from image provider	
retry_count	INTEGER	Number of API retries	– remove
created_at	TIMESTAMP	Job creation time	
updated_at	TIMESTAMP	Last job update	–remove

5. credit_transactions

Column	Type	Key	Description
id	UUID	PK	Transaction ID
user_id	UUID	FK → users.id	User
job_id	UUID	FK → generation_jobs.id, Nullable	Associated generation job
type	VARCHAR	Added / Reserved / Used / Released / Purchased	
credits	INTEGER	Number of credits involved	
balance_after	INTEGER	Available balance after transaction	
status	VARCHAR	Pending / Completed / Failed	
created_at	TIMESTAMP	Transaction timestamp	

- single user single image
- single user multiple image
- multiple users single image
- multiple users multiple images

Scenarios:

1. Same user sent to request simultaneously: Scenario, I need that Credit checking and reserving. Credit should be an atomic property.
2. More number of request like around 100 request simultaneously sent: even though we have a functionality, we cannot send all the request. Once we need to set a batch of size and then send the request as there will be an API request limit or rate limit for the image generation provider API.
3. If provider returns for 429, that is too many request: So let's say we get 429, then we wait then we try with the back off. If you're able to successful. Then continue the process or else if you reach maximum retry reached, then we will consider it as fail and return the reserve credit.
4. User close the browser during generation: In this case, we have Credit reserve plus job created already, the backend should continuously process even if the user close the tab loses internet lock out or refreshes. Then later we have get job status property. and return the output
5. Railway back and crashes for this case, we have to separately Store. The job created so that if the back in crash is also the memory from the Python will be vanished, but we will be able to fetch the ongoing process from the job table.-- already handled by the backend
6. Railways redeploy occurs during generation: for this, as previously mentioned, we have job ID and job state so we can continue the process seamlessly -- already handled by the backend
7. External API generated image but are back and never received the response.: each of the generation should ideally have an ID or reference where supported or your own state should make retry controlled and traceable – this is already taken care by the external api provider
8. Multiple railway applicants update the same wallet

Over all Flow



